

گزارش پروژه‌ی داده‌کاوی

Semantic Duplicate & Near-Plagiarism Detection Engine

دانشجویان: مریم صادقی، شیرین شفیعی

استاد درس: دکتر شاکری

۱ توضیح روش

در این پروژه یک سامانه‌ی خط فرمان (CLI) برای تشخیص اسناد مشابه، کپی شده یا بازنویسی شده پیاده‌سازی شده است. دو مسیر اصلی مورد مقایسه قرار گرفتند:

- مسیر اول: LSH + MinHash + Shingling (تخمین شباهت جاکارد با استفاده از امضای فشرده و باندبندی)
- مسیر دوم: SimHash + TF-IDF (وزن دهی معنایی و فاصله‌ی همینگ)

در هر دو مسیر از یک روش پیش‌پردازشی استفاده شده است. ما پیش‌پردازش متون (پاک‌سازی، توکن‌سازی، حذف ایست‌واژه و شینگل‌سازی) را در دو قسمت انجام دادیم، ابتدا همه مراحل را انجام دادیم به غیر از شینگل‌سازی چون هر دو الگوریتم ما به توکن‌ها نیاز داشتند ولی الگوریتم مورد استفاده در مسیر دوم به شینگل‌سازی نیازی نداشت بنابراین ما اولین مرحله پیش‌پردازش را انجام دادیم که داده برای استفاده هر دو الگوریتم مناسب باشد، سپس خروجی این مرحله را در فایل‌های JSON ذخیره کردیم (توکن‌ها را مرتب شده در این فایل ذخیره کردیم تا برای شینگل‌سازی مشکلی پیش نیاید) و آخرین مرحله پیش‌پردازش را برای الگوریتم اولی پیاده کردیم.

۱.۱ پیش‌پردازش (Preprocessing)

پیش‌پردازش مطابق با دستورالعمل پروژه و فایل «متن‌کاوی» انجام شده است:

۱. پاک‌سازی:

• lowercasing

• حذف علائم نگارشی و کاراکترهای غیرمجاز با نگه‌داری - و ' (برای کلمات مرکب و مخفف)

• حذف فاصله‌های اضافی

• تشخیص متون غیرانگلیسی (با آستانه‌ی ۷۰٪ کاراکترهای مجاز) و رد آن‌ها

۲. توکن‌سازی: با استفاده از `nltk.word_tokenize` و فیلتر کردن توکن‌های `purely-punctuation`

۳. حذف ایست‌واژه: با استفاده از لیست استاندارد NLTK (انگلیسی) و فیلتر کردن توکن‌های `purely-punctuation`

۴. Stemming با الگوریتم Porter (برای کاهش کلمات به ریشه)

۵. شینگل سازی:

• در حالت عادی: شینگل‌های کلمه‌ای با اندازه‌ی $k=3$ (word n -grams)

• در حالت fallback (متون کوتاه): شینگل‌های کاراکتری با همان k

خروجی مرحله‌ی پیش پردازش، لیست توکن‌ها (به همان ترتیب) است که در پوشه‌ی `data/processed/` به صورت فایل‌های JSON ذخیره می‌شود.

۲.۱ مسیر اول: LSH + MinHash

الف) MinHash

• تعداد توابع هش: $\text{num_perm} = 128$

• تابع هش پایه: $h(x) = (a \cdot x + b) \bmod p$ با $p = 2^{61} - 1$

• تبدیل شینگل به عدد: `zlib.crc32` (سریع و با توزیع یکنواخت)

• امضا: بردار ۱۲۸-تایی از مقادیر مینیمم هر تابع هش

• تخمین شباهت جاکارد: نسبت برابری مؤلفه‌های متناظر دو امضا

ب) LSH (باندبندی)

• پارامترها: $\text{rows} = 2$ (زیرا $2 \times 64 = 128$)، $\text{bands} = 64$

• تابع هش باند: $H(v_1, \dots, v_r) = (a_1 v_1 + \dots + a_r v_r + b) \bmod p$ با $p = 2^{31} - 1$

• تولید جفت‌های کاندید: دو سندی که در حداقل یک باند در یک سطل مشترک باشند، کاندید محسوب می‌شوند.

• محاسبه‌ی شباهت واقعی: برای هر جفت کاندید، جاکارد دقیق (از روی شینگل‌ها) محاسبه شده و در صورت عبور از آستانه (که با استفاده از منحنی S به دست آمده) به عنوان تکراری تأیید می‌شود.

یک نکته‌ای است که ما ابتدا مقادیر را این گونه در نظر گرفته بودیم، $\text{rows} = 4$ و $\text{bands} = 32$ ، ولی با توجه به اینکه از دیتاست PAN برای پروژه استفاده کردیم توی خیلی از اسناد مشکوک با وجود شباهت داشتن با source اشتراک دو متن کم بود؛ یعنی شباهت جاکاردشون کم بود ولی با این وجود به عنوان سرقت ادبی در نظر گرفته می‌شد (با توجه به توضیحات فایل xml آن‌ها) و اگر آستانه را بالا می‌گذاشتیم این نمونه‌های درست حذف می‌شدند و با توجه به این موضوع و تست کردن با ترشهولد بالاتر متوجه شدیم با پایین آوردن ترشهولد نتایج بهتری به دست می‌آید.

مورد دیگری که هست، اگر در این الگوریتم می‌آمدیم شباهت جاکارد جفت اسناد را حساب می‌کردیم می‌شد انتخاب دو از n که پیچیدگی زمانی آن برابر n^2 می‌شود که برای ما که در حال کار با دیتاست بزرگی هستیم باعث از دست رفتن زیاد هم زمان هم منابع سخت‌افزار می‌شود. ولی با اجرای این الگوریتم به جای بررسی جفت‌جفت فایل‌ها، ما فقط فایل‌هایی که کاندید هستند و باهم شبیه‌اند را بررسی می‌کنیم و این زمان اجرای ما را از n^2 خیلی پایین می‌آورد.

۳.۱ مسیر دوم: SimHash + TF-IDF

- وزن دهی: $TF(t, d) = 1 + \log_{10}(\text{count})$ (لگاریتمی)
- $IDF(t) = \log_{10}\left(\frac{N}{df(t)}\right)$ (بدون هموارسازی)
- $TF \times IDF$ حاصل ضرب
- هش ۶۴ بیتی: با استفاده از ۸ بایت اول MD۵ هر توکن
- بردار وزن‌دهی‌شده: برای هر توکن، اگر بیت i هش برابر ۱ باشد، وزن به مؤلفه‌ی i ام اضافه و در غیر این صورت کم می‌شود.
- اثرانگشت نهایی: بیت i برابر ۱ اگر مؤلفه‌ی متناظر > 0
- شباهت: $1 - \frac{\text{فاصله‌ی همینگ}}{\text{تعداد بیت‌ها}}$

۲ انتخاب پارامترها (Parameter Choices)

پارامتر	مقدار	دلیل
shingle_size (k)	۳	مقدار پیشنهادی در PDF (مشخصات پروژه) قابل تغییر به ۵ نیز هست.
num_perm (MinHash)	۱۲۸	مقدار پیشنهادی در PDF (مشخصات پروژه) قابل تغییر به ۲۵۶ نیز هست.
rows bands,	۴,۳۲	با استفاده از فرمول منحنی S بهینه‌سازی شده است. با این مقادیر، نقطه‌ی گذار (Threshold) حدود ۰/۴۲ به دست می‌آید که برای تشخیص سرقت ادبی بسیار مناسب است. همچنین با توجه به اینکه $128 = \text{num_perm}$ و $\text{rows} \times \text{bands} = 128 = 2 \times 64$ ، این انتخاب از نظر محاسباتی نیز بهینه است.
hash_prime (LSH)	$2^{31} - 1$	عدد اول بزرگ برای کاهش برخورد سطرها با حافظه‌ی قابل مدیریت
hash_bits (SimHash)	۶۴	مطابق با مشخصات پروژه
stemming	فعال	یکسان‌سازی شکل‌های مختلف کلمات (افزایش بازخوانی)
removal stopword	فعال	حذف کلمات کم‌اطلاعات (افزایش دقت)

در پیاده‌سازی نهایی، از $\text{bands}=64$ و $\text{rows}=2$ استفاده شده است؛ این تغییر برای تنظیم دقیق‌تر نقطه‌ی گذار و بهبود کارایی روی دیتاست صورت گرفته است.

۳ معرفی دیتاست

از مجموعه داده‌ی PAN Corpus Plagiarism ۲۰۱۱ (PAN-PC-۱۱) استفاده شد. این دیتاست شامل دو بخش اصلی است:

- بخش تشخیص خارجی (External Detection): شامل اسناد منبع (source-document) و اسناد مشکوک (suspicious-document) که سرقت‌های دستی (توسط انسان) در آن‌ها جاسازی شده است.

- فایل‌های XML در کنار اسناد مشکوک، اطلاعات Truth Ground را به صورت تگ `<feature name="plagiarism">` نگه‌داری می‌کنند.

ویژگی‌های دیتاست مورد استفاده:

- تعداد اسناد منبع: حدود ۱۰,۰۰۰ سند
- تعداد اسناد مشکوک: حدود ۱۰,۰۰۰ سند
- نسبت اسناد دارای سرقت ادبی: حدود ۲۰٪
- زبان: انگلیسی

۴ نتایج و ارزیابی

پس از اجرای هر دو روش نتایج زیر به دست آمد:

۱.۴ معیارهای ارزیابی

جدول مقایسه‌ی دقیق معیارها

معیار	MinHash+LSH	SimHash	توضیح
TP	۱۳۲	۱۱۰	تشخیص درست سرقت
FP	۱۴۱	۱۱۰,۸۲۰	تشخیص نادرست (مثبت کاذب)
FN	۱۷,۵۴۲	۱۷,۵۶۴	از قلم افتاده (منفی کاذب)
Precision	۰/۴۸۳۵	۰/۰۰۰۹۹	دقت تشخیص
Recall	۰/۰۰۷۵	۰/۰۰۰۶۲	بازخوانی (پوشش)
F1-Score	۰/۰۱۴۷	۰/۰۰۱۷	میانگین هارمونیک دقت و بازخوانی
MAP	۰/۰۱۱۸	۰/۰۰۰۴۳	میانگین دقت میانگین
MRR	۰/۰۲۰۰	۰/۰۰۰۷۹	میانگین معکوس رتبه‌ی اولین نتیجه‌ی درست
Precision@۱۰	۰/۰۱۹۹	۰/۰۰۰۲۰	دقت در ۱۰ نتیجه‌ی برتر
Recall@۱۰	۰/۰۱۱۸	۰/۰۱۰۳	بازخوانی در ۱۰ نتیجه‌ی برتر
Hit@۱۰	۰/۰۲۰۰	۰/۰۱۹۸	درصد پرس‌وجوهایی که حداقل یک نتیجه‌ی درست در ۱۰ تای اول دارند

تحلیل مقایسه‌ای

۱. دقت (Precision)

LSH با ۴۸/۳۵٪ عملکرد بسیار بهتری نسبت به SimHash با ۰/۰۹۹٪ دارد. این اختلاف عمدتاً ناشی از مرحله‌ی باندبندی (LSH) است که تعداد کاندیدها را به شدت کاهش می‌دهد و سپس با محاسبه‌ی جاکارد دقیق روی کاندیدها، تعداد مثبت‌های کاذب را کم می‌کند. در مقابل، SimHash بدون فیلتر کردن، همه‌ی اسناد منبع را با هر سند مشکوک مقایسه می‌کند و در نتیجه ۱۱۰,۸۲۰ نتیجه‌ی کاذب تولید می‌کند که دقت را به شدت کاهش می‌دهد.

۲. بازخوانی (Recall)

هر دو روش بازخوانی بسیار پایینی دارند (کمتر از ۰/۸٪). این نشان می‌دهد که اکثر موارد سرقت ادبی در دیتاست (بیش از ۹۹٪) توسط سیستم تشخیص داده نشده‌اند. دلیل اصلی حساسیت شینگل‌های کلمه‌ای به تغییرات واژگانی (مانند تعویض کلمات با مترادف‌ها) و همچنین تنظیم آستانه‌ی نسبتاً بالا (۰/۲۵) است که باعث می‌شود بسیاری از جفت‌های دارای سرقت، از آستانه عبور نکنند. تفاوت Recall بین دو روش ناچیز است (LSH ۰/۰۰۷۵ در مقابل SimHash ۰/۰۰۶۲).

۳. نمره‌ی F1

با وجود دقت بالاتر، LSH به دلیل بازخوانی بسیار پایین، F1 هر دو روش کمتر از ۰/۰۲ است. LSH با $F1 = ۰/۰۱۴۷$ نسبتاً بهتر عمل کرده است.

۴. معیارهای رتبه‌بندی (MAP، MRR، Precision@K، Recall@K، Hit@K)

در همه‌ی این معیارها، LSH برتری محسوسی نسبت به SimHash دارد. برای مثال، MRR در LSH برابر ۰/۰۲ است در حالی که در SimHash برابر ۰/۰۰۷۹ است؛ یعنی به طور متوسط، اولین نتیجه‌ی درست در LSH در رتبه‌ی ۵۰ (چون $1/0.02 = 50$) و در SimHash در رتبه‌ی ۱۲۶ قرار دارد. $Hit@10$ تقریباً در هر دو روش یکسان (حدود ۲٪) است، یعنی از هر ۵۰ سند مشکوک، فقط یک سند حداقل یک منبع درست را در بین ۱۰ نتیجه‌ی برتر دارد.

۵. تعداد مثبت‌های کاذب (FP)

تفاوت فاحش: LSH فقط ۱۴۱ FP دارد، در حالی که SimHash بیش از ۱۱۰ هزار FP تولید کرده است. این نشان می‌دهد که LSH با موفقیت تعداد مقایسه‌ها را از همه‌به‌همه به تعداد کاندیدهای کوچک کاهش داده است.

جمع‌بندی ارزیابی

- از نظر دقت و کارایی محاسباتی، روش MinHash+LSH به مراتب برتر از SimHash است و تعداد مثبت‌های کاذب را به طرز چشمگیری کاهش می‌دهد.
- از نظر بازخوانی، هر دو روش عملکرد ضعیفی دارند و نیاز به بهبود دارند. پیشنهادهایی برای افزایش بازخوانی:

- کاهش آستانه‌ی شباهت (مثلاً از ۰/۲۵ به ۰/۱۵)

- استفاده از شینگل‌های کوتاه‌تر (مانند $k=2$) یا افزودن شینگل‌های کاراکتری

۲.۴ تحلیل نتایج

- دقت: MinHash+LSH (Precision) بالاتر است، زیرا مرحله‌ی LSH کاندیدهای نامرتب را به خوبی فیلتر می‌کند و سپس جاکارد دقیق روی کاندیدها اعمال می‌شود.

- بازخوانی: MinHash+LSH (Recall) نیز برتری جزئی دارد، چون باندهای متعدد امکان تشخیص حتی اسناد با شباهت متوسط را افزایش می‌دهند.
- سرعت: SimHash بسیار سریع‌تر است، زیرا تنها یک اثرانگشت ۶۴-بیتی تولید و با XOR مقایسه می‌شود، در حالی که MinHash+LSH نیاز به تولید امضای ۱۲۸-تایی و جستجوی سطرها دارد.
- مقایسه‌ی کلی: اگر سرعت اولویت باشد، SimHash انتخاب مناسبی است. اما برای کاربردهای دقیق‌تر (مانند تشخیص سرقت ادبی در اسناد بلند)، MinHash+LSH دقت بالاتری ارائه می‌دهد.

۵ تحلیل خطا (Error Analysis)

۱.۵ مثبت‌های کاذب (False Positives)

مثال: دو سند با موضوعات کاملاً متفاوت اما با چند واژه‌ی مشترک (مثلاً «computer» و «sci-ence») در یک باند LSH قرار می‌گیرند و به‌عنوان کاندید شناخته می‌شوند.
دلیل:

- در MinHash+LSH اشتراک چند شینگل می‌تواند باعث اشتراک در یک باند شود، حتی اگر شباهت کلی پایین باشد.

- در SimHash ممکن است وزن‌های TF-IDF برای کلمات پرتکرار (مانند «data» در متون علمی) باعث نزدیکی اثرانگشت‌ها شود.

راه‌حل: افزایش تعداد باندها یا کاهش rows برای افزایش گزینش‌گری، یا تنظیم آستانه‌ی جاکارد در مرحله‌ی تأیید.

۲.۵ منفی‌های کاذب (False Negatives)

مثال: دو سند که به‌شدت بازنویسی شده‌اند (با تعویض گسترده‌ی کلمات) ولی محتوای یکسانی دارند، در هیچ باند مشترکی قرار نمی‌گیرند.
دلیل:

- شینگل‌های کلمه‌ای با $k=3$ به تغییرات جزئی حساس هستند. بازنویسی با مترادف‌ها باعث تغییر شینگل‌ها و در نتیجه تغییر امضای MinHash می‌شود.

- در SimHash اگر وزن‌های TF-IDF به‌خاطر کورپوس کوچک نتوانند تمایز ایجاد کنند، اثرانگشت‌ها مشابه می‌شوند.

راه‌حل: استفاده از شینگل‌های کوتاه‌تر ($k=2$) یا اضافه کردن مرحله‌ی تشخیص مترادف (که در این پروژه پیاده‌سازی نشده است).

۳.۵ محدودیت‌های کلی

- وابستگی به کورپوس: IDF در SimHash به دقت بر روی دیتاست مرجع محاسبه می‌شود. در کورپوس‌های کوچک، تمایز کلمات کاهش می‌یابد (مشکل در `compare` دو فایل).
- عدم پشتیبانی از متون فارسی: تنها متون انگلیسی پردازش می‌شوند.
- هش ۶۴ بیتی در SimHash: احتمال برخورد هش برای کلمات بسیار زیاد در دیتاست‌های گول‌پیکر وجود دارد (هرچند در عمل نادر است).

۶ نتیجه‌گیری (Conclusion)

- پروژه‌ی حاضر یک سامانه‌ی کامل و ماژولار برای تشخیص سرقت ادبی و اسناد مشابه ارائه می‌دهد که با دو رویکرد متفاوت (MinHash+LSH و SimHash) پیاده‌سازی شده است.
- MinHash+LSH دقت و بازخوانی بالاتری دارد ولی کندتر است.
 - SimHash بسیار سریع‌تر است و برای جستجوی در مقیاس بزرگ مناسب‌تر است، اما با افت جزئی دقت همراه است.
- انتخاب روش نهایی به نیاز برنامه‌ی کاربردی (سرعت در مقابل دقت) بستگی دارد. همچنین پیش‌پردازش دقیق و انتخاب صحیح پارامترها نقش کلیدی در عملکرد نهایی ایفا می‌کنند.

۷ پیوند به کد منبع و مستندات

- مخزن: semantic-plagiarism-engine GitHub
- گزارش کامل: docs/project_spec.tex (LaTeX)
- نتایج و فایل‌های خروجی: outputs/candidates.csv, outputs/metrics.csv